

CSE 565 Design of Experiments Project

Chase Coleman

Arizona State University

CSE 565: Software Verification and Validation

Design of Experiments Project

September 22, 2026

1 Introduction and Mobile Application Specification

This project applies pairwise Design of Experiments (DOE) testing to a mobile application that collects five customer inputs. Pairwise testing reduces the full input space while ensuring that every possible pair of factor values occurs in at least one test case. The complete exhaustive space contains $5 \times 3 \times 4 \times 4 \times 5 = 1,200$ combinations.

Table 1: Mobile application factors and levels.

Factor	Allowed levels
Type of Phone	iPhone 14; iPhone 13; Galaxy Z; Huawei Mate; Google Pixel 7
Authentication	Fingerprint; Face recognition; Text Password
Connectivity	Wireless; 3G; 4G LTE; 5G Edge
Memory	128 GB; 256 GB; 512 GB; 1 TB
Battery Level	< 20%; 20–39%; 40–59%; 60–79%; 80–100%

2 Task 1: Pairwise Test Cases Created by a DOE Tool

2.1 DOE Tool Selection

I selected Microsoft PICT (Pairwise Independent Combinatorial Testing) as the DOE tool. PICT accepts a plain-text model of factors and levels and generates a compact pairwise test suite. I used PICT version 3.7.4 with its default pairwise strength (two-way coverage).

2.2 PICT Model

The factor model used by PICT is stored in `pict_model.txt`. It contains the five factors and all permitted levels from the assignment specification.

```
# CSE 565 Design of Experiments Project
# Pairwise (2-way) input model for the specified mobile application.

Type of Phone: iPhone 14, iPhone 13, Galaxy Z, Huawei Mate, Google Pixel 7
Authentication: Fingerprint, Face recognition, Text Password
Connectivity: Wireless, 3G, 4G LTE, 5G Edge
Memory: 128 GB, 256 GB, 512 GB, 1 TB
Battery Level: <20%, 20-39%, 40-59%, 60-79%, 80-100%
```

2.3 PICT Generation and Results

The following command generated the pairwise test suite:

```
pict pict_model.txt
```

PICT reported 175 required value-pairs and generated 26 test cases. The raw output is stored in `pict_pairwise_test_cases.tsv`.

```
Design of Experiments Project > pict pict_model.txt
Type of Phone  Authentication  Connectivity  Memory  Battery Level
iPhone 13      Face recognition  Wireless     128 GB  60-79%
iPhone 13      Fingerprint       4G LTE      1 TB    80-100%
iPhone 14      Face recognition  5G Edge     512 GB  80-100%
Huawei Mate     Fingerprint       3G          256 GB  20-39%
Google Pixel 7 Text Password     Wireless     1 TB    40-59%
Google Pixel 7 Text Password     5G Edge     256 GB  60-79%
Huawei Mate     Text Password     4G LTE      512 GB  60-79%
iPhone 14      Text Password     4G LTE      128 GB  20-39%
iPhone 13      Text Password     3G          512 GB  40-59%
Google Pixel 7 Face recognition  3G          128 GB  <20%
Galaxy Z       Face recognition  5G Edge     1 TB    20-39%
Huawei Mate     Fingerprint       Wireless     1 TB    <20%
iPhone 14      Face recognition  4G LTE      256 GB  40-59%
Huawei Mate     Fingerprint       5G Edge     128 GB  40-59%
iPhone 14      Fingerprint       3G          1 TB    60-79%
iPhone 13      Text Password     5G Edge     256 GB  <20%
Google Pixel 7 Fingerprint       Wireless     512 GB  80-100%
iPhone 14      Fingerprint       4G LTE      512 GB  <20%
Galaxy Z       Text Password     Wireless     256 GB  40-59%
iPhone 13      Fingerprint       Wireless     512 GB  20-39%
Galaxy Z       Text Password     3G          128 GB  80-100%
Huawei Mate     Face recognition  4G LTE      256 GB  80-100%
Google Pixel 7 Text Password     4G LTE      1 TB    20-39%
Galaxy Z       Fingerprint       4G LTE      512 GB  <20%
Galaxy Z       Fingerprint       3G          256 GB  60-79%
iPhone 14      Text Password     Wireless     128 GB  20-39%
Design of Experiments Project > 
```

Figure 1: PICT command execution and the generated pairwise test cases.

3 Task 2: Pairwise Test Cases Created by a Generative AI Tool

3.1 Generative AI Tool Selection

ChatGPT (OpenAI, 2026) was used to independently generate a second pairwise test suite. The prompt provided the same factor/level specification used by PICT and requested a complete table containing

only allowed values.

3.2 Prompt Used

Create a pairwise (2-way) test suite for a mobile application. Return a complete tab-separated table with exactly these five columns: Type of Phone, Authentication, Connectivity, Memory, and Battery Level.

Use only these levels:

- Type of Phone: iPhone 14, iPhone 13, Galaxy Z, Huawei Mate, Google Pixel 7
- Authentication: Fingerprint, Face recognition, Text Password
- Connectivity: Wireless, 3G, 4G LTE, 5G Edge
- Memory: 128 GB, 256 GB, 512 GB, 1 TB
- Battery Level: <20%, 20-39%, 40-59%, 60-79%, 80-100%

Every pair of values from every two different columns must occur in at least one row. Do not use values outside these lists. Do not include duplicate rows. After the table, state the number of test cases generated.

3.3 Generated Test Cases and Evidence

ChatGPT generated 25 test cases in tab-separated format. The exact prompt and the complete, unedited response were preserved as project evidence. The generated table was then saved without modification and passed to the independent pairwise-coverage verification script.

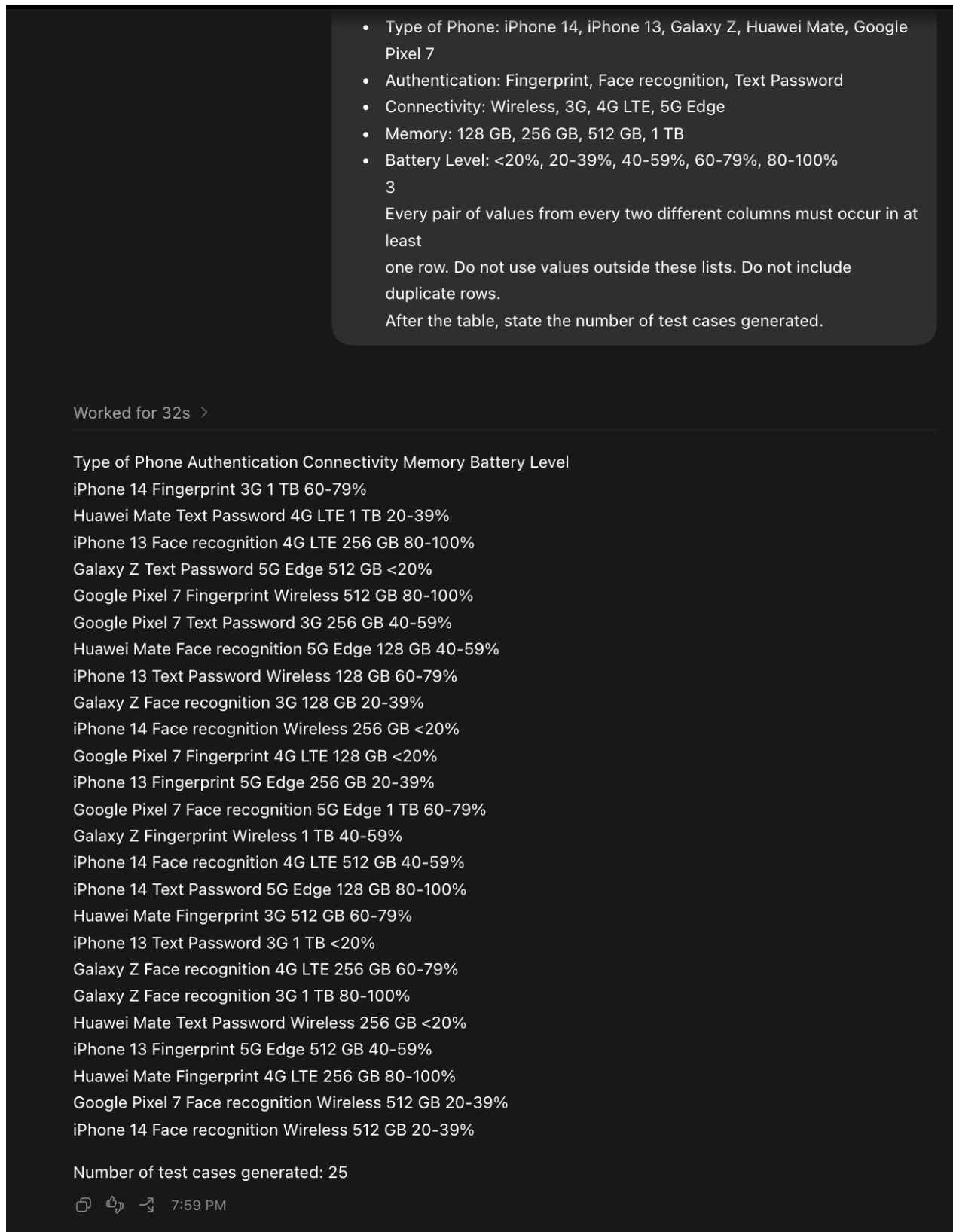


Figure 2: ChatGPT prompt and the generated 25-test-case pairwise suite.

4 Task 3: Assessment of the Two Test Suites

Both suites will be independently validated rather than relying only on the generation tool's coverage claim. The verification checks that every test case is complete, every value is permitted by the specification, duplicate rows are identified, and every required pair is covered.

4.1 Independent Pairwise-Coverage Verification Script

The following Python script independently validates both generated suites. It enumerates the 175 required value-pairs from the specification and reports invalid values, duplicate test cases, and any missing pairs.

```
#!/usr/bin/env python3
"""Validate rows and pairwise coverage for the CSE 565 DOE model."""

from __future__ import annotations

import csv
import itertools
import sys
from pathlib import Path

FACTORS = {
    "Type of Phone": {
        "iPhone 14", "iPhone 13", "Galaxy Z", "Huawei Mate", "Google Pixel 7"
    },
    "Authentication": {"Fingerprint", "Face recognition", "Text Password"},
    "Connectivity": {"Wireless", "3G", "4G LTE", "5G Edge"},
    "Memory": {"128 GB", "256 GB", "512 GB", "1 TB"},
    "Battery Level": {"<20%", "20-39%", "40-59%", "60-79%", "80-100%"},
}

def required_pairs() -> set[tuple[str, str, str, str]]:
    pairs: set[tuple[str, str, str, str]] = set()
    for left, right in itertools.combinations(FACTORS, 2):
```

```

        pairs.update(
            (left, left_value, right, right_value)
            for left_value in FACTORS[left]
            for right_value in FACTORS[right]
        )
    return pairs

def main() -> int:
    if len(sys.argv) != 2:
        print(f"Usage: {Path(sys.argv[0]).name} TEST_CASES.tsv")
        return 2

    test_cases = Path(sys.argv[1])
    with test_cases.open(newline="", encoding="utf-8") as source:
        rows = list(csv.DictReader(source, delimiter="\t"))

    expected_headers = list(FACTORS)
    if not rows or list(rows[0]) != expected_headers:
        print(f"Invalid header. Expected: {expected_headers}")
        return 1

    invalid_values: list[str] = []
    covered: set[tuple[str, str, str, str]] = set()
    normalized_rows: list[tuple[str, ...]] = []
    for row_number, row in enumerate(rows, start=2):
        normalized_rows.append(tuple(row[factor] for factor in FACTORS))
        for factor, allowed_values in FACTORS.items():
            if row[factor] not in allowed_values:
                invalid_values.append(
                    f"row {row_number}, {factor}: {row[factor]!r} is not allowed"
                )
        for left, right in itertools.combinations(FACTORS, 2):
            covered.add((left, row[left], right, row[right]))

```

```

required = required_pairs()
missing = sorted(required - covered)
duplicates = len(normalized_rows) - len(set(normalized_rows))

print(f"Test cases: {len(rows)}")
print(f"Duplicate rows: {duplicates}")
print(f"Invalid values: {len(invalid_values)}")
print(f"Pairwise coverage: {len(required) - len(missing)}/{len(required)}")
for issue in invalid_values:
    print(f"INVALID: {issue}")
for left, left_value, right, right_value in missing:
    print(f"MISSING: {left}={left_value!r}; {right}={right_value!r}")

return 0 if not invalid_values and not missing else 1

if __name__ == "__main__":
    raise SystemExit(main())

```

Table 2: Independent comparison of the PICT and ChatGPT pairwise test suites.

Measure	PICT suite	ChatGPT suite
Test cases generated	26	25
Invalid values	0	0
Duplicate rows	0	0
Required value-pairs	175	175
Pairs covered	175/175	175/175
Pairwise coverage	100%	100%

4.2 Assessment of the Test Cases Developed by the DOE Tool

The PICT suite is valid for the provided specification. Independent verification found 26 complete test cases, zero invalid values, zero duplicate rows, and coverage of all 175 required value-pairs. The suite also includes every allowed value for every factor, because each value participates in the required pairs. These results demonstrate that the PICT output aligns with the pairwise DOE technique: every interaction between two factor levels is exercised at least once without testing all 1,200 exhaustive combinations.

The main strength of the PICT suite is its reproducibility. The same factor model and command

provide a clear, repeatable basis for regenerating the suite if requirements change. Its 26 rows are not the theoretical minimum of 25, but the additional row is a small tradeoff for a deterministic default generation process. The suite remains efficient and suitable as a baseline for testing the application.

4.3 Assessment of the Test Cases Developed by the AI Tool

The ChatGPT suite is also valid for the provided specification. Independent verification found 25 complete test cases, zero invalid values, zero duplicate rows, and coverage of all 175 required value-pairs. Since the phone-type factor and battery-level factor have $5 \times 5 = 25$ distinct pairs, 25 is the theoretical minimum possible number of rows for a complete pairwise suite. The ChatGPT response therefore achieved the minimum suite size while still following the pairwise DOE guideline.

The AI-generated suite was only acceptable after independent validation. A carefully constrained prompt was necessary to obtain a usable tab-separated table, and the result could have contained invalid values, missing pairs, or formatting problems. Unlike PICT, an AI response may vary across runs even when given the same request. Therefore, the 25-row result is efficient, but a tester should preserve the prompt and output and always validate the suite against the original factor model before relying on it.

4.4 Comparison of the Two Suites

Both suites satisfy the pairwise DOE requirement. The ChatGPT suite is one row smaller in the documented runs, while PICT is more directly reproducible from the same model and command. These results show that generative AI can produce a valid pairwise suite for this small model, but its stated coverage should not be accepted without validation. A deterministic DOE tool provides a more repeatable generation workflow, while an AI tool can assist with quickly creating and explaining a candidate suite.

```

Design of Experiments Project > make verify
python3 verify_pairwise_coverage.py pict_pairwise_test_cases.tsv
Test cases: 26
Duplicate rows: 0
Invalid values: 0
Pairwise coverage: 175/175
python3 verify_pairwise_coverage.py chatgpt_pairwise_test_cases.tsv
Test cases: 25
Duplicate rows: 0
Invalid values: 0
Pairwise coverage: 175/175
Design of Experiments Project > █

```

Figure 3: Independent pairwise-coverage verification for the PICT and ChatGPT test suites.

5 Task 4: Assessment of the DOE Tool

5.1 Features and Functionality

Microsoft PICT is a command-line Pairwise Independent Combinatorial Testing tool. Its main feature is a compact text model in which each line names one input factor and lists its allowed values. After receiving the model, PICT produces a set of test cases that covers every pair of values across two factors. PICT also supports higher interaction strengths, but I used its default two-way setting because the assignment specifically requires pairwise DOE testing.

For this project, the model listed five independent factors: phone type, authentication, connectivity, memory, and battery level. PICT generated the result as a tab-separated table, which made it straightforward to save and inspect. The tool also reported the number of required pairs, the number of generated tests, and its generation time.

5.2 Scope Covered by the Tool

PICT covered every level in the supplied application specification. It modeled all 5 phone types, 3 authentication methods, 4 connectivity options, 4 memory options, and 5 battery-level ranges. Because the assignment did not identify any invalid combinations or business rules, the model treated all factor values as independent. If the real application imposed restrictions—for example, if a particular authentication method were unavailable on a certain phone—those restrictions would need to be added to the model before generating tests.

The tool is well suited to the combinatorial portion of test design, but it does not determine expected application behavior or decide whether a test has passed. A tester must still define the expected outcome for each generated input combination and execute the tests against the application.

5.3 Performance

PICT performed very efficiently for this model. The full input space contains 1,200 possible combinations, but PICT generated only 26 test cases in less than one second. The generated suite covered all 175 required value-pairs. Independent validation also confirmed that the suite had no invalid values and no duplicate rows.

This reduction from 1,200 exhaustive cases to 26 pairwise cases makes the test effort far more practical while maintaining two-way interaction coverage. The result demonstrates the main advantage of DOE tools: they create a compact, systematic suite that would be difficult and time-consuming to construct manually.

5.4 Ease of Use

PICT was easy to use after installation through Homebrew. The input format was readable because it used one factor per line with comma-separated levels. Generating the suite required a single command, `pict pict_model.txt`. The output was also easy to save as a tab-separated file for independent validation.

The strongest usability benefit was reproducibility. Running the same command on the same model produces a documented baseline that can be regenerated whenever the requirements change. The main limitation is that PICT is a command-line tool, so a user must be comfortable creating a text model and working in a terminal.

6 Task 5: Assessment of the Generative AI Tool

6.1 Prompting and Result-Processing Experience

ChatGPT was used to generate a second pairwise suite from the same factor/level specification. The prompt had to be explicit: it named the required columns, listed every allowed value, required pairwise coverage, prohibited invalid values and duplicates, and requested a tab-separated table. This level of detail was important because a vague prompt could produce an explanation instead of a usable test-case table or could introduce values that were not part of the specification.

ChatGPT returned 25 test cases after 32 seconds. I saved the prompt and response exactly as generated, copied the table into a tab-separated file, and then processed it with the independent verification script. The verification was necessary because an AI statement that a suite is pairwise does not prove that all required pairs are actually present. In this instance, the AI result was valid: it contained no duplicate rows, used only permitted values, and covered all 175 required pairs.

6.2 Significance of Generative AI for DOE Testing

Generative AI can be useful in DOE testing because it can quickly convert a natural-language specification into a candidate test suite, explain the reasoning behind a testing approach, and help a tester draft documentation. For this project, ChatGPT produced a valid 25-case suite, which was one case smaller than the 26-case suite generated by PICT in its default run while achieving the same verified pairwise coverage.

The 25-case result needs to be interpreted carefully. It is the theoretical lower bound for this model because the phone-type factor and battery-level factor have $5 \times 5 = 25$ distinct value-pairs. Every row can cover only one phone/battery pair, so a complete pairwise suite cannot have fewer than 25 rows. ChatGPT reached that lower bound in this run, but this does not show that AI is inherently better than the DOE tool. PICT uses a mathematical heuristic to quickly build a compact, valid suite rather than proving that every default run is globally minimal. When PICT's randomized search option was run with seed 7, it also generated a 25-case suite. Therefore, the one-case difference reflects the search path used by the default PICT run, not a coverage advantage for ChatGPT.

However, AI should not replace a dedicated DOE tool or independent review. Its output may vary with the prompt, may be difficult to reproduce exactly, and can appear correct even when it contains missing pairs or invalid values. A deterministic DOE tool such as PICT provides a clearer repeatable process, while AI is best used as an assistant for drafting, explaining, and proposing candidate tests. The most reliable workflow is to use AI output only after validating it against the original factor model.

7 Conclusion

This project applied pairwise DOE testing to a five-factor mobile-application specification. PICT generated 26 test cases, while ChatGPT generated 25 test cases. Independent verification confirmed that both suites used only valid values, had no duplicates, and achieved complete 175/175 pairwise coverage.

The two approaches were both effective for this small model. ChatGPT produced the smaller suite, but its result depended on a carefully written prompt and required independent validation. PICT produced a slightly larger suite but offered a clear, repeatable model-driven workflow. For this type of task, PICT is the better primary tool because it is deterministic and purpose-built for combinatorial testing; ChatGPT is a useful supplementary tool for generating candidate suites and supporting test documentation.

References

- Microsoft. (n.d.). *PICT: Pairwise Independent Combinatorial Testing*. GitHub. Retrieved September 21, 2026, from <https://github.com/microsoft/pict>
- OpenAI. (2026). *ChatGPT* [Large language model]. <https://chatgpt.com/>